

# SentryOps

## Autonomous Multi-Agent Production Incident Response & Root Cause Analysis Platform

### OPERATIONAL DOMAIN

- **Site Reliability Engineering (SRE):** Autonomous incident triage and dynamic mitigation.
- **Unified Observability:** Tri-modal correlation across metrics, logs, and traces.
- **Enterprise Safety:** Rigorous human-in-the-loop approvals before infrastructure change.

### CORE ARCHITECTURE

- **Agent Orchestration:** Python 3.11+ with LangGraph state graph coordination.
- **Frontend & Storage:** React 19, TypeScript, PostgreSQL 15 + pgvector for RAG.
- **Infrastructure:** Kubernetes cluster (Kind) with Prometheus, Loki, Jaeger & Grafana.

### GOVERNING PRINCIPLE

- **Empowered AI Investigation:** Parallel investigative reasoning across telemetry stores.
- **Strict Deterministic Guardrails:** Hardcoded safety rules, zero direct cluster actuation.
- **Anti-Hallucination Gate:** Evidence IDs verified against ground truth before RCA generation.

# The Modern Production Incident Dilemma

Operational Bottlenecks in Modern Cloud-Native Microservices

## FRAGMENTED TELEMETRY

- **Alert Storms & Noise:** High-severity incidents trigger cascading alerts across dozens of decoupled microservices.
- **Context Switching:** On-call engineers waste critical minutes manually correlating logs across Loki, metrics across Prometheus, and traces across Jaeger.
- **Siloed Insights:** Lack of unified causality mapping delays diagnosis.

## PROLONGED MTTR

- **80%+ Diagnosis Overhead:** In complex distributed architectures, identifying the root cause consumes the majority of incident recovery time.
- **Runbook Retrieval Latency:** Relevant operational runbooks and historical postmortems are rarely surfaced when outages strike.
- **Cognitive Burnout:** High-stress manual triage increases operator mistakes under severe SLA pressure.

## UNCONSTRAINED AI RISKS

- **Hallucinated Commands:** Giving an LLM direct shell access or raw kubectl execution creates catastrophic reliability risks.
- **Cascading Outages:** Unbounded autonomous remediation can accidentally delete databases or terminate healthy services.
- **Missing Auditability:** Autonomous black-box actions violate compliance, governance, and organizational change controls.

# The SentryOps Solution

End-to-End Autonomous Incident Lifecycle with Deterministic Safety Guardrails

- **Simulated Production Architecture:**

- 5 FastAPI microservices (`order`, `payment`, `notification`, `user`, `api-gateway`).
- Dynamic fault-injection API running inside the Kubernetes cluster on port 8005.

- **End-to-End Observability Pipeline:**

- Full OpenTelemetry instrumentation across all routes.
- Real-time Prometheus metrics, Loki structured log streaming, and Jaeger distributed traces visualized in Grafana.

- **Parallel Multi-Agent Orchestration:**

- LangGraph manages specialized agents with structured fan-in synthesis.

- **Deterministic Safety Guardrails:**

- **Strict Allowlist:** Remediations limited exclusively to `RESTART\_SERVICE`, `SCALE\_SERVICE`, and `ROLLBACK\_SERVICE`.
- **Non-LLM Risk Engine:** Deterministic rules classify actions (`LOW`, `MEDIUM`, `HIGH`) independently of LLM reasoning.
- **Mandatory Human Approval Gate:** All medium and high-risk proposals halt at `PENDING\_APPROVAL`.
- **Zero Cluster Actuation:** Agents operate against `SIMULATED\_INFRA\_STATE` without direct `kubectl` privilege.
- **Automated Verification & Postmortem:**
  - Post-remediation health checks verify recovery and compile structured postmortems.

# Multi-Agent Parallel Investigation

LangGraph Fan-Out / Fan-In Investigative Workflow & Anti-Hallucination Gates

## • Incident Trigger & Dispatch:

- Outage detected via error-rate threshold or synthetic fault API.
- `IncidentManager` spawns parallel investigation across 6 specialized agents.

## • Parallel Specialized Agents (Fan-Out):

- **LogAgent**: Queries Loki for error stacks, 5xx status codes, and exceptions.
- **MetricAgent**: Queries Prometheus for error rate spikes and P95 latency.
- **TraceAgent**: Queries Jaeger for error spans, tags, and trace waterfalls.
- **DeploymentAgent**: Checks recent config changes, releases, and fault flags.
- **InfraAgent**: Scrapes node memory, CPU, and pod restart health.
- **KnowledgeAgent**: Retrieves relevant historical runbooks via pgvector RAG.

## • Structured Synthesis (Fan-In):

- Specialized agents populate a centralized incident evidence store.

## • RCA Agent & Anti-Hallucination Gate:

- Performs evidence linkage and cross-source causal synthesis.
- **Strict Validation Gate**: The RCA Agent algorithmically validates that all cited evidence IDs exist in the evidence store before conclusions are finalized.

## • RemediationAgent & Risk Policy:

- Formulates structured mitigation action matching the RCA hypothesis.
- Enforces deterministic risk assessment before any action can proceed.

# Deterministic Risk Policy & Approval Gate

Human-in-the-Loop Operational Safety Classification

## LOW RISK POLICY

- **Policy Rule:** Restarting a single unhealthy microservice instance.
- **Autonomous Execution:** Allowed (Simulated).
- **Operator Action:** None required (automatically advances to post-remediation verification).

## MEDIUM RISK POLICY

- **Policy Rule:** Scaling service replica counts (`SCALE\_SERVICE`).
- **Autonomous Execution:** Prohibited.
- **Operator Action:** Mandatory **Operator Approval** required via dashboard before execution.

## HIGH RISK POLICY

- **Policy Rule:** Multi-instance restarts or configuration rollbacks (`ROLLBACK\_SERVICE`).
- **Autonomous Execution:** Prohibited.
- **Operator Action:** Mandatory **Operator Approval** required.

## • State Machine Enforcement:

- Any proposal classified as `MEDIUM` or `HIGH` automatically transitions to `PENDING\_APPROVAL` and halts execution.
- **Operator Approval API** (`POST /incidents/{id}/approve`):
  - Records approval audit trail, transitions status to `APPROVED`, and triggers deterministic simulated execution.
- **Operator Rejection API** (`POST /incidents/{id}/reject`):
  - Transitions status to `REJECTED`, cancels remediation execution, and returns incident to `INVESTIGATING`.

# Observable Telemetry & Operator Dashboard

Real-Time Telemetry Visibility Across Prometheus, Loki, Jaeger & React 19

## • Grafana Observability Dashboard:

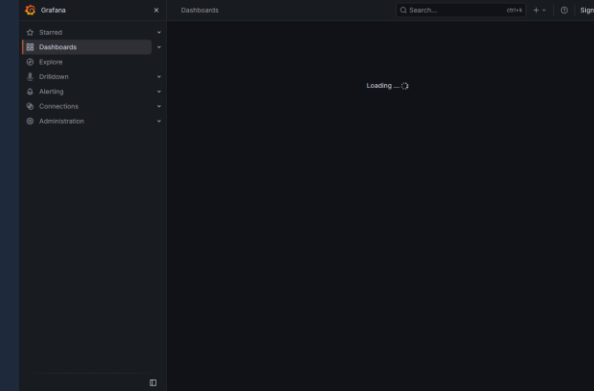
- **Service Health Status:** Real-time status panel tracking health across all 5 microservices (`order`, `payment`, `notification`, `user`, `api-gateway`).
- **Request Rate:** Per-second throughput across services.
- **Error Rate (5xx):** Instant visibility into error spikes during active faults; returns deterministically to 0 upon recovery.
- **P95 Latency:** Histogram quantiles tracking delays.
- **Live Service Logs:** Real-time Loki log streaming filtered by `service\_name`.

## • Jaeger Distributed Tracing:

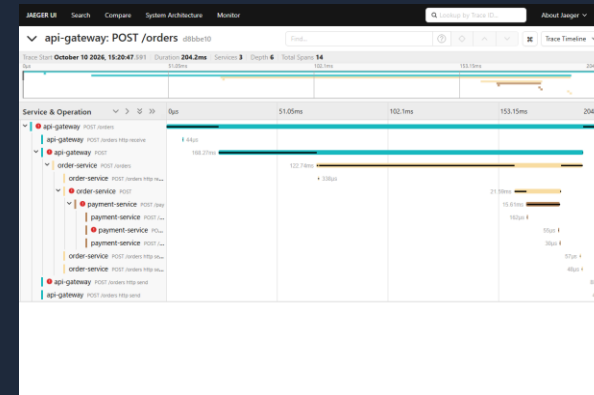
- End-to-end trace waterfalls capturing cross-service HTTP calls and pinpointing error spans.

## • React 19 SRE Incident Dashboard:

- Interactive incident timeline, evidence inventory, RCA summary, human approval card, and postmortem view.



Grafana: Error Rate (5xx) Spike During Active Fault



Jaeger: Distributed Trace Waterfall Pinpointing Error Span

# Live Demonstration Walkthrough

Validated End-to-End Scenario: `http_500_spike` on `payment-service`

## • Incident & Fault Identifiers:

– Incident ID: `INC-CD90E2` | Fault Injection ID: `FAULT-1D8A4DA5`.

## • Quantitative HTTP Traffic Verification:

– **Baseline:** 10/10 OK (100% success, avg latency: 1.116s).

– **Fault Active:** 0/10 OK, 10/10 500s (100% failure, avg latency: 1.008s).

– **Fault Cleanup:** 10/10 OK (100% recovered, avg latency: 1.051s).

## • Harvested Telemetry Evidence:

– `EV-TRACE-3F77AC`: 10 Jaeger traces analyzed (7 error spans identified).

– `EV-CHANGE-9964BD`: Active synthetic fault metadata detected.

– `EV-INFRA-5B1A7A`: Prometheus node/pod infrastructure health verified.

– `EVID-KNOW-6750`: 3 relevant historical runbooks retrieved from pgvector.

## • Remediation Proposal & Risk Gate:

– Proposal: `SCALE_SERVICE` for `payment-service` (replicas: 2).

– Labeled: `[DEMO FIXTURE - MOCK_LLM]`.

– Safety Gate: Classified as `MEDIUM` risk -> Halts at `PENDING_APPROVAL`.

The screenshot displays the SENTRYOPS Autonomous Incident Response Platform interface. At the top, the header shows 'SENTRYOPS' and 'Autonomous Incident Response Platform'. Below the header, a navigation bar includes a 'Back to Incidents' button. The main content area features a large incident card for 'INC-CD90E2', labeled 'RESOLVED' and 'HIGH', titled 'Payment Service HTTP 500 Spike Incident'. This card includes a table with columns for Service, Fault Type, and Created, showing 'payment-service', 'http\_500\_spike', and '10/10/2026, 09:50:51' respectively. Below the incident card, there are four panels: 'Service Health: payment-service' showing a 'healthy' status with 2 replicas and current version; 'Root Cause Analysis' showing a 'Mocked Root Cause' with 90% confidence and evidence tags; 'Timeline' showing a sequence of events from incident creation to agent start; and 'Remediation & Risk' showing a 'SCALE\_SERVICE' proposal with a deterministic rationale and a JSON configuration snippet.

| Service         | Fault Type     | Created              |
|-----------------|----------------|----------------------|
| payment-service | http_500_spike | 10/10/2026, 09:50:51 |

| Status  | Replicas | Version |
|---------|----------|---------|
| healthy | 2        | current |

**Root Cause Analysis**

Mocked Root Cause

Confidence: 90%

EVID-TRACE-3F77AC EVID-KNOW-6750 EV-INFRA-5B1A7A EV-CHANGE-9964BD

**Timeline**

- INCIDENT\_CREATED  
10/10/2026, 09:50:51  
Incident created
- AI\_INVESTIGATION\_STARTED  
10/10/2026, 09:50:52  
AI\_INVESTIGATION\_STARTED
- AGENT\_STARTED  
10/10/2026, 09:50:52

**Remediation & Risk**

SCALE\_SERVICE

[DEMO FIXTURE - MOCK\_LLM] Deterministic proposal for demonstration: Scale service replicas to 2 to alleviate traffic/latency load.

```
{ "replicas": 2, "target_service": "payment-service" }
```

React 19 SRE Dashboard: Remediation Proposal & Risk Assessment

# Execution, Verification & Postmortem

Deterministic Lifecycle Completion & Evidence Verification

## • Human Operator Approval Gate:

– Operator submitted approval via dashboard (`POST /incidents/{id}/approve`).

## • Simulated Execution Safety Boundary:

– Execution status: `SUCCESS` against in-memory `SIMULATED\_INFRA\_STATE`.

– Zero Kubernetes cluster mutation: `kubectl get deployment payment-service` remained strictly at 1 replica.

## • Deterministic Post-Remediation Verification:

– Check 1 (`replica\_count`): 2 replicas verified (Passed).

– Check 2 (`service\_health`): Healthy status verified (Passed).

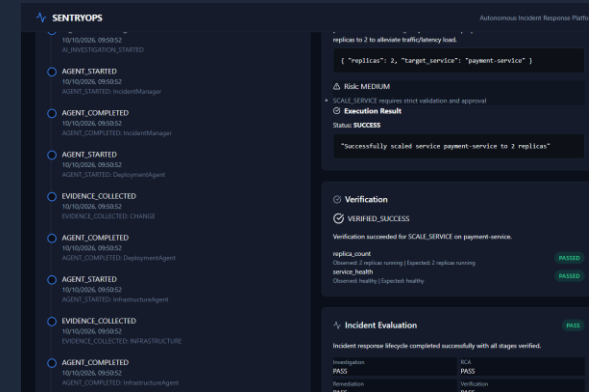
– Outcome: Marked `VERIFIED\_SUCCESS` -> Incident resolved.

## • Incident Replay & Evaluation:

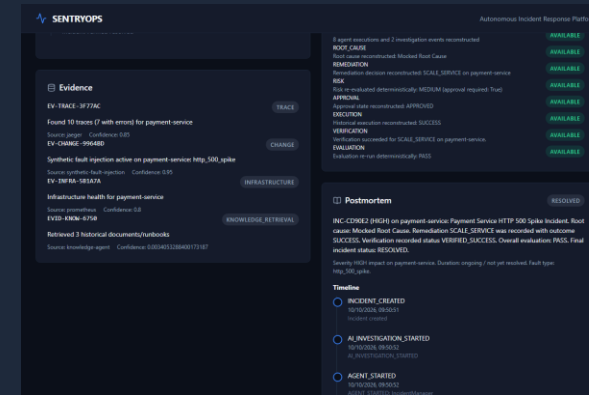
– Evaluation Framework: `PASS` across all 4 dimensions (Investigation, RCA, Remediation, Verification).

– Replay Consistency: `MATCH` (verified without historical re-execution).

– Postmortem: 9 structured sections generated deterministically.



Approval Gate & Deterministic Health Verification



Automated 9-Section SRE Incident Postmortem View



# Technical Testing & Metric Disclosure

Evidence-Based Verification Across Test Suites & Build Pipelines

- **Kubernetes-Compatible Test Suites:**

- **152 / 152 PASSED (100% Pass Rate)** in 27.78 seconds.
- Covers Phase 7 Knowledge Agent, Phase 8 Workflow Demo, Phase 8 Remediation, Phase 9 Verification, Phase 11 Evaluation, Phase 12 Replay/Postmortem, and Phase 13 Kubernetes architecture.

- **Zero Failures, Zero Skipped.**

- **Frontend Production Build:**

- ``npm run build`` completed cleanly in 1.96s with zero errors or warnings.

- **Full Repository Test Suite Disclosure:**

- **169 Passed, 15 Failed** out of 184 total tests.
- **Root Cause of 15 Failures:** Exclusively legacy Docker Compose integration tests asserting obsolete host port mappings (``BACKEND_API = 8080`` vs ``8000``, and direct host ports ``8001-8004`` which are internal Kubernetes ClusterIPs).
- **Zero Code Regressions** across all core application logic and agent orchestration workflows.

# Security Architecture & Operational Boundaries

Hardened Production Guardrails & Local Prototype Disclosures

- **Hardened Production Constraints:**

- **No Pod Privilege Escalation:** All application pods configure ``automountServiceAccountToken: false``.
- **Zero RBAC Abuse:** No Kubernetes cluster-admin or service account permissions granted to application pods.
- **No Real Cluster Actuation:** LLM agents have zero access to shell commands or live Kubernetes APIs.
- **Allowlist Enforcement:** Actions strictly restricted to 3 deterministic operations (``RESTART``, ``SCALE``, ``ROLLBACK``).

- **Disclosed Local Development Boundaries:**

- **Authentication / Authorization:** FastAPI backend has no token-based authentication middleware (scoped for local prototype demonstration).
- **Default Credentials:** Development manifests configure default database passwords (``sentryops:sentryops``).
- **Insecure Metrics TLS:** Kind Metrics Server configured with ``--kubelet-insecure-tls`` for local container scraping.

# Summary & Future Roadmap

Key Achievements & Next Steps for Production Evolution

## • Key Project Achievements:

- Complete, observable, multi-agent incident response lifecycle from alert to postmortem.
- **100% passing** Kubernetes-compatible test suites (152/152).
- Deterministic safety gates preventing AI hallucination and unauthorized actuation.
- Full-stack integration with Prometheus, Loki, Jaeger, Grafana, and React 19 UI.

## • Future Strategic Roadmap:

- **Production LLM Integration:** Connecting live models (Claude 3.5 Sonnet / GPT-4o) under the same deterministic safety gates.
- **Enterprise Security:** Multi-tenant RBAC and API key authentication on incident management endpoints.
- **GitOps-Driven Actuation:** PR-based remediation proposals via ArgoCD or Flux.
- **Cloud Observability:** Persistent storage provisioned on AWS EKS or GCP GKE.